

PROGRAMMATION FONCTIONNELLE 2 : TD11 FONCTIONS D'ORDRE SUPÉRIEUR

mai 2026 — Pierre Rousselin

On rappelle la définition des fonctions `fold_left` et `fold_right`, vues en cours :

```
let rec fold_right f l accu =
  match l with
  | [] -> accu
  | a::l -> f a (fold_right f l accu)

let rec fold_left f accu l =
  match l with
  | [] -> accu
  | a::l -> fold_left f (f accu a) l
```

Remarque : ces fonctions sont celles de la bibliothèque standard d'OCaml (dans le module `List`), l'ordre des arguments est celui qui a été choisi dans cette bibliothèque.

Exercice 1 : sur fold

1. Donner le type de `fold_left` et le type de `fold_right`.
2. Évaluer pas à pas :
 - a) `fold_right ( + ) [10; 42; 3] 0`
 - b) `fold_left ( + ) 0 [10; 42; 3]`
3. Définir la fonction `repeat` telle que `repeat s n` est la chaîne `s` répétée `n` fois.
4. Laquelle des deux expressions suivantes est bien typée? Quelle est sa valeur?
 - a) `fold_right repeat [2; 3; 4] "foo"`
 - b) `fold_left repeat "foo" [2; 3; 4]`
5. Soit la fonction suivante :

```
val count_letter : string -> int -> int
(** count_letter s n retourne le nombre d'occurrences de la lettre
    numéro n dans la chaîne s.
    La lettre numéro 0 est 'a', la lettre numéro 25 est 'z',
    les lettres de numéro <= 0 sont des 'a' et
    les lettres de numéro >= 25 sont des 'z'
    Par exemple, (count_letter "foo bar baz" 1) vaut 2. *)
```

Laquelle des deux expressions suivantes est bien typée? Si oui, quelle est sa valeur?

- a) `fold_right count_letter ["barbazbar"; "bouh"; "pacha"; "zazie"] 25`
- b) `fold_left count_letter 25 ["barbazbar"; "bouh"; "pacha"; "zazie"]`

--- * ---

Correction de l'exercice 1 :

1. `val fold_left : ('a -> 'b -> 'a) -> 'a -> 'b list -> 'a`
`val fold_right : ('a -> 'b -> 'b) -> 'a list -> 'b -> 'b`
2. `repeat` est de type `string -> int -> string` donc il y a une erreur de type avec `fold_right`.
 Ensuite, on a :


```
fold_left repeat "foo" [2; 3; 4]
= fold_left repeat "foofoo" [3; 4]
= fold_left repeat "foofoofoofoofoofoo" [4]
= "foofoofoo...foo" (24 fois)
```

3. `count_letter` est de type `string -> int -> int`, donc cette fois `fold_right` qui est du bon type. Il y a deux « z » dans « zazie », un « c » dans « pacha », un « b » dans « bouh » et trois « b » dans « barbazbar ». Donc
- ```
fold_right count_letter ["barbazbar"; "bouh"; "pacha"; "zazie"] 25 = 3.
```

--- \* ---

## Exercice 2 : fold pour fabriquer des fonctions

En utilisant l'une des fonctions `fold`, définir (sans itérer sur les listes) les fonctions suivantes :

- le maximum d'une liste d'entiers (l'entier minimal est `Int.min_int`);
- la longueur d'une liste;
- le nombre d'occurrences d'un certain entier dans une liste d'entiers;
- la fonction `flatten` qui prend une liste de listes; et retourne la liste obtenue en « aplatissant » cette liste de listes, par exemple `flatten [[1; 2]; [3]; [4; 5]]` devrait valoir `[1; 2; 3; 4; 5]`;
- la fonction `all` telle que `all p l` vaut `true` lorsque tous les éléments de `l` satisfont le prédicat `p`;
- la fonction `any` telle que `any p l` vaut `true` lorsqu'au moins un des éléments de `l` satisfait le prédicat `p`;
- (en utilisant une fonction intermédiaire), le maximum d'une liste avec un type option (le maximum de la liste vide est `None`)<sup>1</sup>

--- \* ---

## Correction de l'exercice 2 :

- `let max_list_int l = fold_left max Int.min_int l`
- `let len_with_fold l = fold_left (fun i _ -> i + 1) 0 l`
- `let nb_occ n l = fold_left (fun i h -> if (h = n) then i + 1 else i) 0 l`
- `let all p l = fold_left (fun b h -> b && p h) true l`
- `let any p l = fold_left (fun b h -> b || p h) false l`
- `let max_option x y =`  
`match x with`  
`| None -> Some y`  
`| Some v -> if y > v then Some y else x`  
  
`let max_list l = fold_left max_option None l`

--- \* ---

---

1. en OCaml, l'opérateur `<` est polymorphe, de type `'a -> 'a -> bool`